

NestJS Feature Enhancement Assignment

Objective:

The purpose of this assignment is to evaluate your understanding of NestJS, REST API design, and your ability to work with existing codebases. You are expected to build a small feature using NestJS based on boilerplate.

Repository:

Fork and clone the following NestJS boilerplate repository to get started:

<https://github.com/nestjs/typescript-starter>

Requirements:

1. Familiarize yourself with the boilerplate project structure and the included features.
2. Your task is to add a new feature to manage a collection of events. Each database entity should have the following properties (will be persisted in **database(eg mysql, postgres, etc)**):
 - **Event Entity**
 - id (unique identifier, auto-generated)
 - title (string, required)
 - description (string, optional)
 - status (enum, required: ['TODO', 'IN_PROGRESS', 'COMPLETED'])
 - createdAt (date, auto-generated)
 - updatedAt (date, auto-generated)
 - startTime
 - endTime
 - invitees (list of Users)
 - **Users Entity**
 - Id
 - name
 - events (list of strings)
3. Design a REST API to perform the following operations on tasks:
 - Create a new task
 - Retrieve a task by its id
 - Delete a task by its id
 - **MergeAll** - a method to merge all overlapping events below to a user. For those event, will invite all members in each event. Overlapping example: E1: 2pm-3pm, E2: 2:45pm-4pm => E_merged: 2pm-4pm
4. Use appropriate NestJS modules, providers, and decorators to implement the feature.
5. Write unit tests and integration tests for your feature, ensuring adequate test coverage.
6. Create a **pull request** to *your forked repo*

Deliverables:

Please provide a link to your GitHub repository containing your solution:

1. The modified NestJS project with the new feature.
2. A README.md file describing how to run the project and tests.
3. Record a short video for demo:
 - a. the 3 APIs for task work
 - b. unittests work

Evaluation Criteria:

Your submission will be evaluated based on the following criteria:

1. Adherence to NestJS best practices and conventions.
2. Code readability and organization.
3. Implementation of the REST API and the task management feature.
4. Test coverage and quality.

FAQ:

1. Should I establish a local database to test the API, or use some mock test instead of a real database connection?

should do both

2. Should the API just return the list of merged events for a user, or will it actually change the list of events in that User entity, then in addition, change the Event table (delete unused events and replace them with the new merged event)?

it will change the database for both entities

3. How to deal with other attributes of the merged Event entity? For example, two overlapping events with different titles, descriptions, and status.

you can append string values and pick a reasonable value for status.